(Fine-tuned BERT) Encoder Comparison for Document Classification

Knowledge Processing in Intelligent Systems: Practical Seminar
Knowledge Technology, WTM
Department of Informatics, University of Hamburg

Sabrina Ebert
sabrina.ebert@studium.uni-hamburg.de

Muhammad Hamza
muhammad.hamza-2@studium.uni-hamburg.de

***Abstract*—Encoder models like BERT are small and cheap compared to large language models and are often used for classifying texts. LoRA makes fine-tuning them cheaper by training only a few small added matrices. It comes with several settings, such as rank, learning rate and which layers it is added to, and it is not clear to us which of them matter most for a small encoder. In this work we fine-tune BERT-base with LoRA for classifying sentences, short news texts and full documents (SST-2, AG News, 20 Newsgroups) and measure if and for which settings the classification performance changes, and by how much. We change one setting at a time, run every setting with three seeds and compare accuracy and macro F1 to a frozen BERT baseline. Our working hypothesis is that learning rate and LoRA placement change the performance more than the rank does. If time allows, we also take a first look inside the fine-tuned models with explainable AI methods, e.g. word attributions, to explain the results.**

I. Introduction and Motivation

- encoders small and cheap vs. LLMs
- LoRA: many settings, little guidance

II. Theoretical Background

A. BERT

B. LoRA

Fine-tuning changes a weight matrix $W_0 \in \mathbb{R}^{d\times k}$ into $W_0 + \Delta W$. LoRA assumes that this change needs only a few directions, a low "intrinsic rank", and writes it as the product of two thin matrices [5]:

$$h = W_0 x + \tfrac{\alpha}{r} BAx, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{1}$$

W_0 stays frozen, only A and B are trained (Fig. 1). A starts random and B at zero, so training starts exactly from the pretrained model. α/r scales the update. Hu et al. set α once and do not tune it [5, Sec. 4.1]. After training, BA can be added to W_0, so the model is as fast as before.

The rank r is the number of independent patterns the update can hold: $BA = \sum_{j=1}^{r} b_j a_j^\top$, one column of B times one row of A each (Fig. 2). With $r = 1$, every row of ΔW is a multiple of the same row $a_1^\top$, so the weights

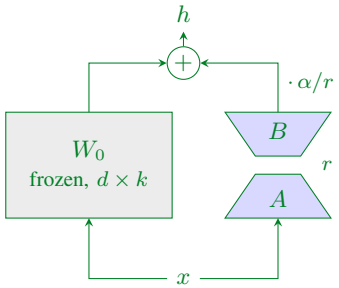


Figure 1. LoRA: the frozen weight W_0 gets a trainable low-rank path BA next to it. Only the blue part is trained. After [5], Fig. 1.

can only change along one direction. For one 768×768 attention matrix of BERT-base [1] and $r = 8$, LoRA trains $2 \cdot 768 \cdot 8 = 12{,}288$ instead of $589{,}824$ weights, about 2%. With LoRA on query and value in all 12 layers plus the classification layer, 296,450 weights are trained, 0.27% of the model. What is left to choose: the rank r, the scale α, which matrices and which layers get LoRA, and the usual training settings.

Which layers. Lower BERT layers encode word types and syntax, higher layers semantic relations such as roles and coreference [13, Sec. 4]. Fine-tuning changes mostly the top layers, and freezing all but the last few costs little [9, Sec. 5]. For classification the last layer gives the best features [12, Tab. 3]. So LoRA can be restricted to the last blocks, e.g. layers 10–12 (`layers_to_transform=[9,10,11]`), which trains 75,266 instead of 296,450 weights at $r = 8$. At the same budget, top layers clearly beat bottom layers [15, Fig. 1b]. Which layers are needed depends on the task [17], so we test it.

C. Related Work

III. Experimental Setup

- Kaggle, SST-2, AG News, 20 Newsgroups
- baseline, LoRA study, other datasets

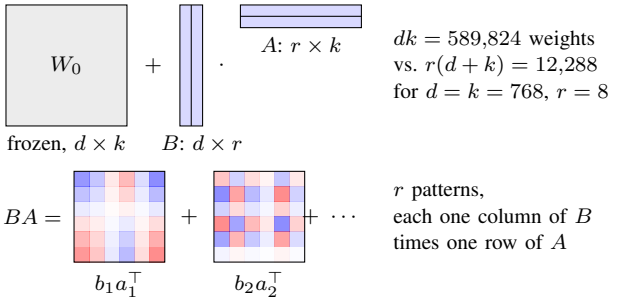


Figure 2. The LoRA update as matrices. Top: the frozen W_0 gets BA added, two thin matrices with r columns and rows. Bottom: BA is a sum of r simple patterns. Each pattern changes every row of W_0 in the same direction, only scaled (blue = up, red = down). A higher rank allows more different directions.

- libraries: transformers, peft, scikit-learn, maybe captum (XAI)
- 3 seeds per setting

A. *Datasets*

Three datasets, from single sentences to whole documents (Fig. 3). Dev is used for all choices, test is scored once at the end.

- GLUE: benchmark of 9 English sentence tasks, test labels are hidden and scored on a server [14]. We use one task, SST-2.
- SST-2: movie review sentences, positive or negative [11]. Train has 67,349 items, mostly short phrases cut out of sentences (median 7 words), validation 872 full sentences. We split validation into dev and test.
- AG News: news title and description, 4 topics (world, sports, business, science and technology), 30,000 train and 1,900 test texts per topic [16]. Median 50 tokens, under 1% longer than 128.
- 20 Newsgroups: forum posts from 20 groups, 11,314 train and 7,532 test, mail headers removed [6]. Median 132 tokens, half longer than 128, 9% longer than 512, so text length matters here.

B. *Classifier on top*

BERT gives one vector of 768 numbers per token. A class needs one vector per text: the [CLS] vector or the mean over all tokens. A linear layer turns it into one score per class, softmax into probabilities, and cross entropy is the loss [1].

- baseline: BERT frozen, only a logistic regression on the mean vector is trained
- LoRA: the linear layer is trained together with LoRA in the chosen layers, all BERT weights stay frozen (Fig. 4)
- weights that change, $r = 8$ on query + value, 2 classes: 294,912 LoRA + 1,538 classifier
- Hugging Face's classifier first passes [CLS] through BERT's pretrained pooler layer, which stays frozen

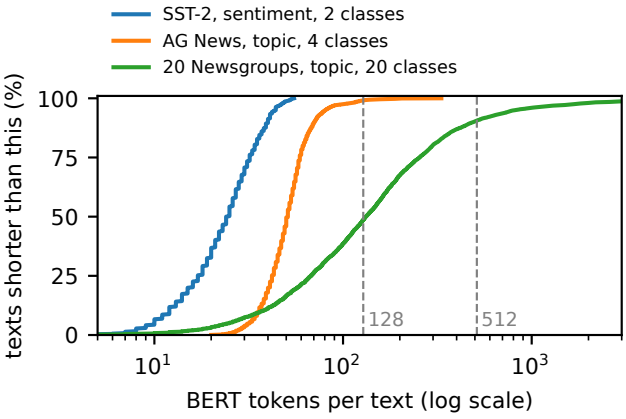


Figure 3. Text length of the three datasets: SST-2 sentences (validation), AG News and 20 Newsgroups (3000 random training texts each). AG News stays below 128 tokens, half of the 20 Newsgroups posts are longer, about one in ten longer than 512.

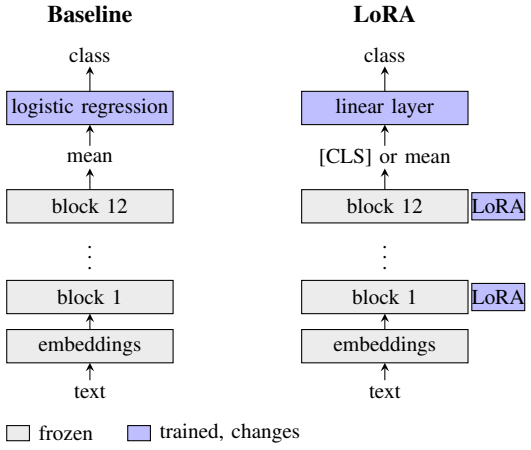


Figure 4. The two setups. Grey parts stay as pretrained, only blue parts change during training. LoRA layers and modules are the settings we vary.

C. *Metrics*

For N dev texts with true class y_i, the predicted class is $\hat{y}_i = \arg\max_c (W v_i + b)_c$, trained with cross entropy.

$$\text{acc} = \frac{1}{N} \sum_{i=1}^{N} [\hat{y}_i = y_i], \qquad R_c = \frac{\#\{\hat{y}_i = y_i = c\}}{\#\{y_i = c\}} \tag{2}$$

R_c is the accuracy within class c, P_c the same among texts predicted as c. Macro F1 averages $F1_c = 2P_cR_c/(P_c + R_c)$ over the C classes, so small classes count as much as large ones. Every value is the mean over 3 seeds, changes are given in points against the reference.

D. *Settings to try, from earlier work*

Sorted from where LoRA sits to how it is trained. Values are the ones reported, mostly for other models.

- layers: LoRA only in layers 10–12 nearly as good as all, only 1–3 much worse, 88.60 vs. 77.87 on MNLI-m, same budget, DeBERTaV3-base [15, Fig. 1b]
- layers: full fine-tuning with all but the top layers frozen keeps most of the accuracy [9, Sec. 5.2]
- layers: about 30% of the layers enough, which ones depends on the task, decoder [17]
- modules: feed-forward 89.99 vs. query only 88.58, MNLI-m, same budget [15, Fig. 1a]
- modules: query + value better than one matrix with higher rank, all four attention matrices best on MNLI, 91.7, GPT-3 [5, Tab. 5]
- modules: all matrices better than two on all 8 GLUE tasks, on SST-2 only by 0.11 [15, Tab. 14]
- modules: all attention vs. query + value at $r = 8$: 59.2 vs. 53.4 exact match, T5-small [10]
- rank: $r = 1$ to 64 on query + value within 0.5 points, GPT-3 [5, Tab. 6]
- rank: $r = 2$ and $r = 8$ both 94.95 on SST-2, DeBERTaV3-base [15, Tab. 1]
- rank: $r = 8$ vs. 32 on SST-2: 93.46 vs. 93.58, RoBERTa-base [7]
- rank: no gain from higher rank on a small BERT, rank 1 almost no loss [3, Sec. 4.4]
- rank: only very small budgets lose, SST-2 94.38 at 0.08% vs. 94.95 at 0.16% of the weights [15, Tab. 5]
- alpha: $\alpha = 2r$ common, e.g. 16 for $r = 8$ [5, Tab. 9], set once and not tuned [5, Sec. 4.1]
- learning rate: $4 \cdot 10^{-4}$ to $5 \cdot 10^{-4}$ on GLUE, $5 \cdot 10^{-4}$ on SST-2, RoBERTa-base [5, Tab. 9], far above $2 \cdot 10^{-5}$ to $5 \cdot 10^{-5}$ for full fine-tuning [1]
- learning rate: most sensitive setting, Llama-3.1-8B [4]
- training: batch 16, warmup 6%, 60 epochs on SST-2, RoBERTa-base [5, Tab. 9]
- pooling: mean better than [CLS] for frozen BERT [8, Tab. 1], no results with LoRA found
- text length: last 510 tokens better than first, IMDb test error 5.44 vs. 5.63, head + tail 5.42 [12, Tab. 2]

IV. Results

V. Discussion and Outlook

Acknowledgments and Use of Tools and AI

We used Claude Code (Anthropic) to set up the LaTeX layout, to suggest some of the references to draft the abstract from our milestones, to draft the LoRA and classifier sections with their sketches and the list of settings from the cited papers, and to improve grammar and wording. The topic, study design and working hypothesis are our own. We checked all text and references and take full responsibility for the content.

REFERENCES

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[2] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. arXiv:2002.06305, 2020.

[3] Daniel Frees, Aditri Bhagirath, and Moritz Bolling. Exploring efficient learning of small BERT networks with LoRA and DoRA. arXiv:2508.17586, 2025.

[4] Haonan He, Jingqi Ye, Minglei Li, Zhengbo Wang, Mengqi Li, Tao Chen, Lei Bai, and Peng Ye. A unified study of LoRA variants: Taxonomy, review, codebase, and empirical evaluation. arXiv:2601.22708, 2026.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[6] Ken Lang. NewsWeeder: Learning to filter netnews. In *International Conference on Machine Learning (ICML)*, 1995.

[7] Shiwei Li, Xiandi Luo, Haozhao Wang, Xing Tang, Ziqiang Cui, Dugang Liu, Yuhua Li, Xiuqiang He, and Ruixuan Li. Beyond higher rank: Token-wise input-output projections for efficient low-rank adaptation. arXiv:2510.23123, 2025.

[8] Xiaofei Ma, Zhiguo Wang, Patrick Ng, Ramesh Nallapati, and Bing Xiang. Universal text representation from BERT: An empirical study. arXiv:1910.07973, 2019.

[9] Amil Merchant, Elahe Rahimtoroghi, Ellie Pavlick, and Ian Tenney. What happens to BERT embeddings during fine-tuning? In *Proceedings of the Third BlackboxNLP Workshop*, pages 33–44, 2020.

[10] Mahendra Singh Rathor and Anagheem Azzam. How small can you go? a controlled study of LoRA rank, target modules, and quantization trade-offs for text-to-SQL on a 60m-parameter model. arXiv:2607.25583, 2026.

[11] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP*, pages 1631–1642, 2013.

[12] Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. How to fine-tune BERT for text classification? In *China National Conference on Chinese Computational Linguistics (CCL)*, 2019.

[13] Ian Tenney, Dipanjan Das, and Ellie Pavlick. BERT rediscovers the classical NLP pipeline. In *Proceedings of ACL*, 2019.

[14] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations (ICLR)*, 2019.

[15] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations (ICLR)*, 2023.

[16] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, 2015.

[17] Fei Zuo, Yizhou Huang, and Kezhi Wang. Mechanistic insights into LoRA: Layer sparsity for adaptive fine-tuning via path patching. arXiv:2610.34756, 2026.

APPENDIX: CODE